Remaining Useful Life Prediction on NASA C-MAPSS FD001 with Engineered Degradation Features

Sarvesh Srinath, Trishit Swarnakar, Aryan Kumar, Husain Khorakiwala and Navodit Sharma
Team T081, Section E, CSAI Project 7

***Abstract*—We predict the remaining useful life (RUL) of turbofan engines in the FD001 subset of NASA C-MAPSS from their multivariate sensor histories. Rolling means, standard deviations and least squares slopes over windows of 5, 10 and 20 cycles, together with a learned health index, are computed within each engine. Elastic Net, Random Forest and XGBoost are tuned with five-fold cross validation grouped by engine on RUL capped at 125 cycles, and the 100 test engines are scored once, at their last observed cycle, after every choice is frozen. The selected XGBoost model reaches a test RMSE of 15.36 cycles and a late-stage RMSE of 7.03 cycles on engines within 50 cycles of failure. Against test labels clipped at 125, Random Forest and XGBoost reach RMSE 14.02 and 14.08, improving on published Random Forest and gradient boosting results and trailing the best published deep models by 0.8 to 1.5 cycles. Our margin over the published tree ensembles comes mainly from the engineered features, and Random Forest and XGBoost cannot be separated on the test set.**

***Index Terms*—remaining useful life, prognostics, C-MAPSS, feature engineering, gradient boosting, random forest, SHAP**

I. Introduction

Maintenance of aircraft engines is either scheduled at fixed intervals or triggered by observed condition. Fixed intervals replace parts that still have life left, while waiting for a fault risks a failure in service. Predicting the remaining useful life (RUL) of an engine, the number of operating cycles left before it fails, lets maintenance be planned close to the point where it is actually needed. The cost of an error is asymmetric: predicting too much remaining life risks an in-service failure, while predicting too little only brings maintenance forward.

The NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) data [1] are the standard public benchmark for this task. Each engine is simulated from a healthy state with unknown initial wear, and 21 sensors and three operating settings are recorded once per cycle. Training engines run to failure; test engines are cut off at an unknown point, and the true RUL is given only at their last observed cycle. We work on the FD001 subset, which has one operating condition and one fault mode.

Our approach keeps the learners simple and puts the effort into the features and the protocol. From the 14 informative sensors in FD001 we compute, within each engine, rolling means, standard deviations and least squares slopes over three window lengths, and a health index learned by linear regression. Elastic Net, Random Forest and XGBoost are trained on RUL capped at 125 cycles and tuned with five-fold cross validation grouped by engine, so that no engine contributes rows to both sides of a split. The test set is used once, after every hyperparameter and the choice of model are frozen.

The contributions of this report are:

- an engine-level evaluation protocol for FD001 in which feature statistics are refitted inside every fold and the test set is scored once, with errors reported separately for engines near failure;
- a comparison of three regressors on raw and engineered features, showing that the engineered features and the move from linear to tree models each improve the RMSE by several cycles, while the two tree ensembles cannot be separated;
- an analysis of what the tree ensembles rely on, using held-out grouped permutation importance and SHAP values, and a comparison of the test results with published FD001 models under matched label clipping.

Section II reviews prior work on C-MAPSS. Section III describes the data and the choice of sensors and target cap, and Section IV the evaluation protocol. Section V defines the engineered features. Section VI covers the models, their tuning and what they rely on, and Section VII reports the test results. Section X discusses the findings and their limits, and Section XI concludes with the plan for sequence models.

II. Related Work

A. Benchmark and target

C-MAPSS simulates run-to-failure trajectories of aircraft turbofan engines with modelled damage propagation [1]. Its FD001 subset is the simplest of the four: one operating condition, one fault mode (high-pressure compressor degradation), and 100 training and 100 test engines. Training engines run until failure; each test engine stops at an unknown point before failure and its true RUL at that cycle is given separately.

A linear RUL target implies that an engine degrades at a constant rate from its first cycle, which the sensors do not show. Heimes [2], who estimated RUL with recurrent neural networks, proposed the piecewise-linear target used in much of the later work [3], [4]: the target is held at a constant maximum early in life and falls linearly to zero at failure. The maximum is chosen from the data and differs between subsets [3]; Ragab et al. use 125 cycles for FD001 and FD003 and 130 for FD002 and FD004 [4]. We cap training targets at 125.

TABLE I

REPRESENTATIVE FD001 METHODS. RMSE VALUES ARE THOSE COLLECTED IN [4], TABLE III, EXCEPT SVR, WHICH IS FROM [3]. OURS IS SCORED AGAINST TEST LABELS CLIPPED AT 125, AS IN TABLE VI. CAP IS THE MAXIMUM OF THE TRAINING TARGET. INPUTS OF THE RANDOM FOREST AND GRADIENT BOOSTING BASELINES ARE FROM [10]; THE DEEP LSTM CAP IS AS RESTATED BY THE SAME AUTHORS IN [11]. A DASH MARKS A SETTING NOT STATED IN THESE SOURCES.

Method	Input	RMSE	Cap
SVR [3]	–	20.96	–
Random Forest [6]	14 sensors, 30 cycles	17.91	–
Grad. boosting [6]	14 sensors, 30 cycles	15.67	–
2D CNN [3]	sensors, 15 cycles	18.45	yes
Deep LSTM [7]	sensor sequence	16.14	130
1D CNN [9]	14 sensors, 30 cycles	12.61	125
ATS2S [4]	14 sensors, 30 cycles	12.63	125
XGBoost (ours)	engineered	14.08	125

B. Classical models

Early data-driven methods for this benchmark include similarity-based prognostics [5]. In the experiments of Babu et al. [3], support vector regression reached an RMSE of 20.96 cycles on FD001, ahead of relevance vector regression (23.80) and a multilayer perceptron (37.56). Zhang et al. [6] report 17.91 for a Random Forest and 15.67 for gradient boosting, against 15.04 for their ensemble of deep belief networks. These tree ensembles are the closest published counterparts of our models.

C. Deep sequence models

Deep models take a window of normalised sensor readings as input and learn their own features. Babu et al. [3] applied a convolutional network along the time axis of 15-cycle windows (RMSE 18.45). Zheng et al. [7] used a long short-term memory (LSTM) [8] network, whose gated memory cells can carry information over long time lags (16.14). Li et al. [9] used a one-dimensional CNN with multiple input channels; at 12.61 it has the lowest FD001 RMSE in the comparison of Ragab et al. [4]. Ragab et al. add an attention mechanism to an LSTM encoder-decoder over 30-cycle windows of 14 sensors and reach 12.63, with the lowest score S (Section IV) in that comparison. Table I summarises these methods; Table VI in Section VII places our test results among them.

D. Gap addressed

The most accurate FD001 results come from models that learn features from raw sensor windows. This work asks how far hand-built degradation features take classical models. For each of the 14 informative sensors we compute the rolling mean, standard deviation and least squares slope over 5, 10 and 20 cycles, after operating regime normalisation, and keep the raw readings, the cycle number and a regression-based health index (Section V). Elastic Net, Random Forest and XGBoost are trained on these features and tuned with cross validation grouped by engine, so that no engine appears in both the training and validation folds and the validation error is not inflated by near-duplicate cycles (Section IV). Sequence models of the kind discussed above are the subject of the second phase of the project.

III. DATASET AND EXPLORATORY ANALYSIS

A. Dataset and quality checks

We use the FD001 subset of NASA C-MAPSS, a simulated turbofan degradation benchmark with one nominal operating condition and one fault mode: high-pressure compressor degradation [1]. Each observation contains an engine identifier, a cycle number, three operating settings and 21 sensor readings. The training set contains 20,631 observations from 100 engines run to failure. All sensor and target decisions use this training set. A final subsection adds descriptive test plots approved in review; no earlier choice was revisited.

We found no missing cells, duplicate rows or duplicate engine-cycle keys. Cycles start at one and are consecutive within each engine, so no imputation or row removal was needed. Recorded lifetimes range from 128 to 362 cycles, with median 199 and mean 206.31 cycles. Figure 1 shows the distribution of lifetimes, counting each engine once, and the RUL distribution, counting every cycle. The latter weights longer histories more heavily. These descriptive distributions are not model evaluation results.

B. Operating settings and sensor selection

Settings 1 and 2 vary slightly around zero, while setting 3 is exactly 100. The scatter plots in Fig. 2 are consistent with the single nominal operating condition specified by NASA. A noisy cloud alone does not establish a physical regime count; no regime clustering is fit here.

We compute sample standard deviation, range, distinct-value count and dominant-value fraction for each sensor. Raw standard deviations are in different physical units, so an absolute variance threshold would not give a fair comparison. Sensors $s_1, s_5, s_{10}, s_{16}, s_{18}, s_{19}$ each have one unique reading and are removed as exact constants. Tiny nonzero standard deviations for some constants are floating-point roundoff.

We also remove s_6, whose only recorded levels are 21.60 and 21.61, consistent with quantization or rounding at the recorded precision. Its standard deviation is approximately 0.001389: 20,225 readings are 21.61 and 406 are 21.60. The 98.03% dominant share is descriptive, not a selection threshold. This FD001 choice matches the 14-sensor set listed by Liu et al. [12], Section 4.2; it does not prove zero predictive information. The resulting 14 retained sensors are $s_2, s_3, s_4, s_7, s_8, s_9, s_{11}, s_{12}, s_{13}, s_{14}, s_{15}, s_{17}, s_{20}, s_{21}$. Sensor s_{17} is quantized but has 13 distinct readings and is retained. Figure 3 summarizes this evidence; the same list is exported as `USEFUL_SENSORS` for feature engineering.

C. Degradation trends and the target cap

For training engine i, let T_i be its final recorded cycle. The uncapped target at cycle t is $r_{i,t} = T_i - t$, calculated from the complete history. Figure 4 aligns five engines to failure. Engines 9, 44, 65, 76 and 100 are sampled without replacement with seed 42. Faint curves are raw readings; solid curves use 11-cycle trailing means computed within each engine, with shorter windows at the start, solely for display. Several selected sensors show clearer directional changes near

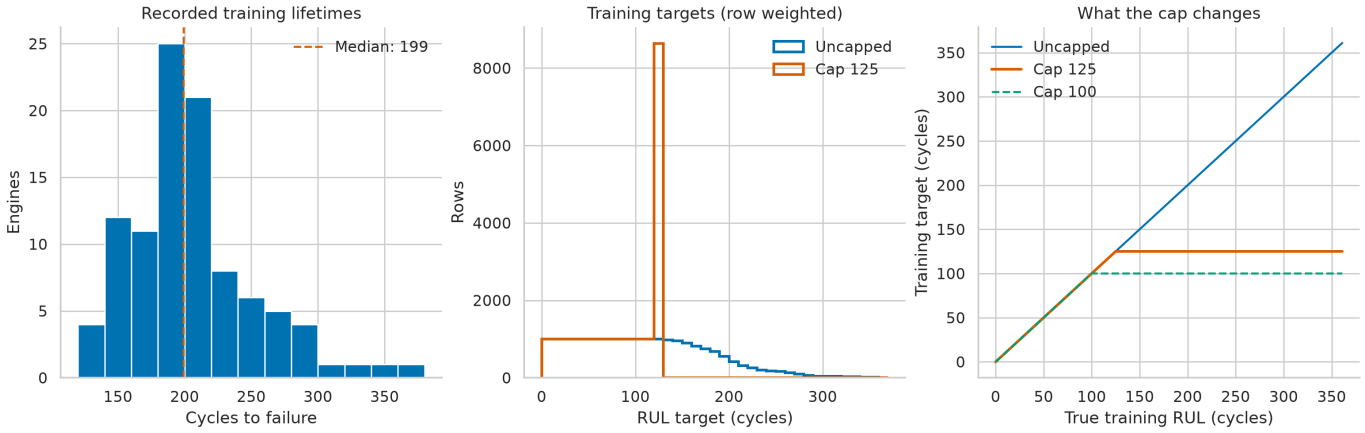


Fig. 1. Training lifetimes, uncapped and capped training targets, and the effect of caps of 100 and 125 cycles. The cap changes labels, not the number of observations.

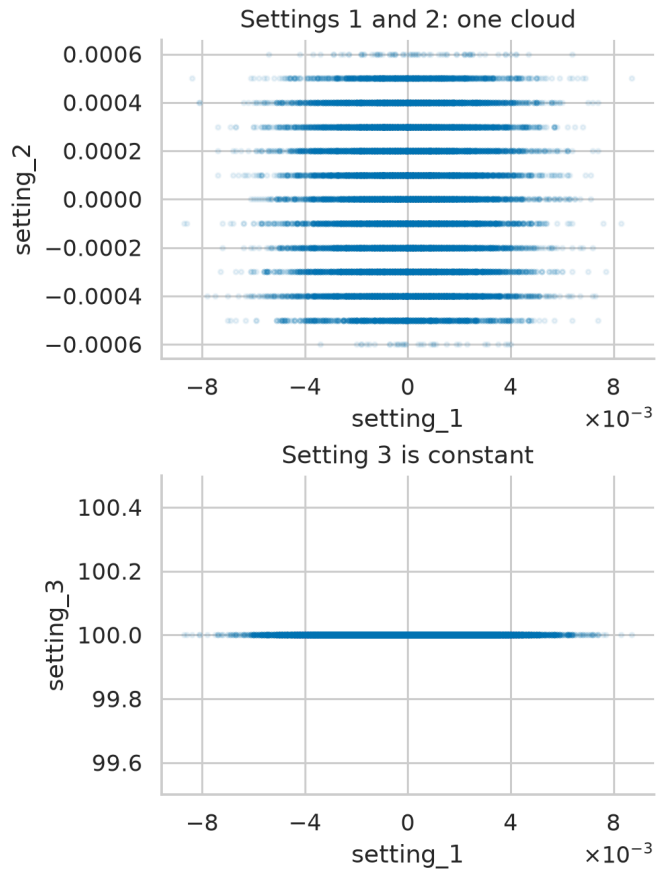


Fig. 2. Training operating settings: a cloud around zero for settings 1 and 2, and a constant value for setting 3.

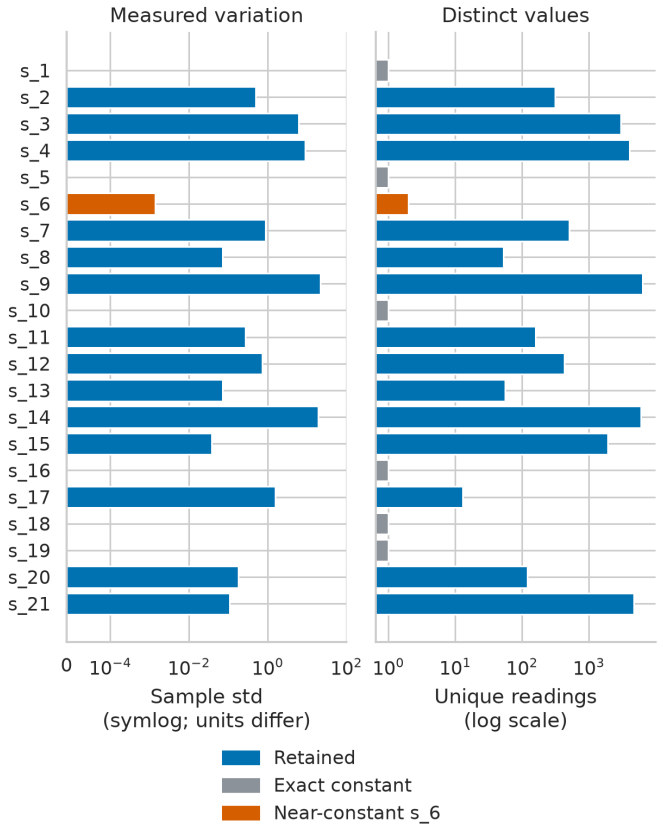


Fig. 3. Sensor variation and distinct-value counts. Exact constants are shown at zero standard deviation; sensor units differ, so bar lengths in the left panel are not directly comparable measures of usefulness.

failure than earlier in life, with differing engine baselines and substantial noise.

We adopt the training target $y_{i,t} = \min(125, r_{i,t})$. Liu, Wang and Yu [12], Section 4.2, use an upper target limit of 125 on C-MAPSS because early sensor evolution carries weak degradation information. Our plots support this motivation but do not prove that early RUL is unlearnable or that 125 is optimal. The cap changes 8,031 training labels (38.93%). A cap of 100 creates a longer plateau and removes more early-life target resolution; no cap preserves exact targets but requires distinguishing more early-life states. The modeling stage compares capped and uncapped targets with engine-

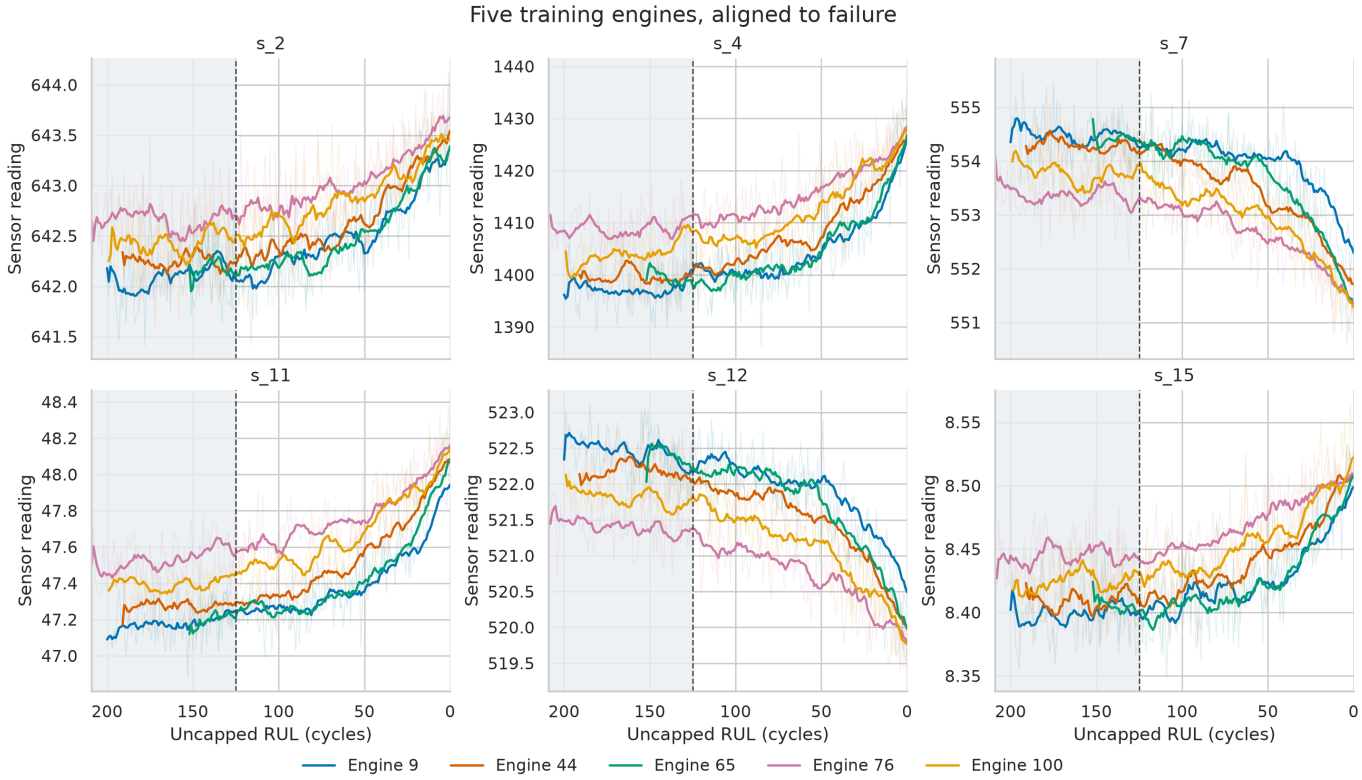


Fig. 4. Six representative retained sensors from five training engines, aligned on uncapped RUL. Failure is at the right. The shaded area has RUL above 125; the dashed line marks the team's target cap.

grouped cross-validation. True test RUL remains uncapped.

D. Correlation and limitations

Figure 5 reports Pearson correlations of the 14 retained sensors and both uncapped and 125-capped training RUL. Several sensors share substantial variation. We do not remove a sensor solely for correlating with another; its model-specific contribution requires subsequent evaluation. These are pooled, row-weighted descriptive associations: adjacent cycles are dependent and between-engine differences contribute. They do not demonstrate causation or independent-sample significance, and weak linear association does not exclude nonlinear or temporal information.

This analysis is specific to FD001 and simulated run-to-failure histories. Sensor selection should be revisited for other operating conditions. Any learned scaler or feature transformation used in cross-validation must be fit within its training fold. The notebook retains its outputs and exports the evidence tables to `results/eda_*.csv` for reproducibility.

E. Final descriptive test diagnostics

After all preceding training decisions, we plot each test engine's last observed cycle and the distribution of its supplied true RUL (Fig. 6). These are descriptive only; no earlier choice was revisited. Last observed cycle is a truncation point, not failure or physical engine age. Labels remain uncapped. No tuning, sensor selection or model evaluation uses these plots.

IV. Evaluation Protocol

A. Splitting by engine

Each training engine in FD001 contributes one row per operating cycle, and consecutive cycles of the same engine differ only by a small amount of degradation and sensor noise. If rows were split at random, cycle t of an engine could fall in the training set while cycle $t+1$ fell in the validation set. The model would then be scored on near copies of rows it had already seen, and the validation error would understate the error on a new engine. We therefore split by engine: every row of a given `unit` lies on one side of the split only. Hyperparameters are tuned with five fold `GroupKFold` cross validation grouped on `unit`, which assigns engines to folds deterministically without shuffling. A single holdout split, used for quick checks, draws 20% of the engines with seed 42. The test set is used once, after all hyperparameters are frozen.

B. Scoring on test engines

Training engines run to failure, but each test engine is cut off at an unknown point before failure, and the dataset gives the true RUL only at its last observed cycle. Each test engine is therefore scored once, using the prediction at its final cycle, so all 100 test engines carry equal weight.

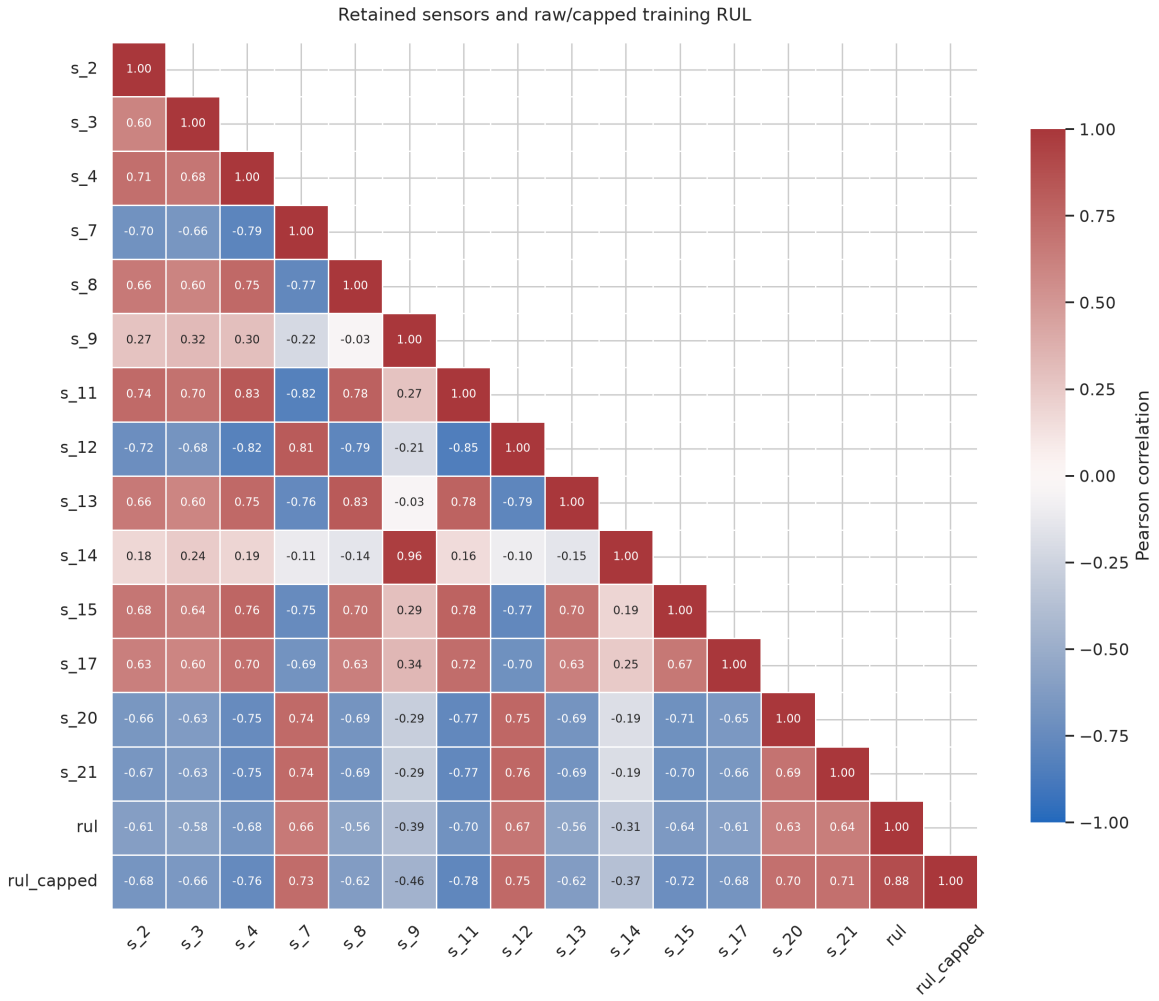


Fig. 5. Pearson correlations of retained sensors with each other and uncapped and 125-capped training RUL. Constants are excluded because their correlations are undefined.

C. Metrics

Let y_i be the true RUL of engine i, $\hat{y}_i$ the prediction, $d_i = \hat{y}_i - y_i$ the error and n the number of engines. We report

$$\mathrm{RMSE} = \sqrt{\tfrac{1}{n}\textstyle\sum_i d_i^2}, \qquad \mathrm{MAE} = \tfrac{1}{n}\textstyle\sum_i |d_i|, \tag{1}$$

$$S = \sum_i s_i, \quad s_i = \begin{cases} e^{-d_i/13} - 1, & d_i < 0, \\ e^{d_i/10} - 1, & d_i \geq 0, \end{cases} \tag{2}$$

where S is the asymmetric scoring function of the PHM08 challenge [1]. RMSE penalises large errors more than MAE does, and both treat early and late predictions alike. The score S does not: a prediction that overestimates RUL by 10 cycles costs as much as one that underestimates it by 13, and the penalty grows exponentially. Overestimating RUL means the engine is believed healthy when it is close to failure, which risks an in service failure; underestimating it only brings maintenance forward. Because S is a sum rather than a mean, it is compared only across models scored on the same set of engines.

D. Early and late stage error

Maintenance decisions are made near the end of an engine's life, so error there matters more than error far from failure. We split the test engines by true RUL into a late stage ($y_i \leq 50$ cycles) and an early stage ($y_i > 50$) and report RMSE and MAE for each. A model with a good overall RMSE but a poor late stage RMSE would be of little use for scheduling maintenance.

V. Degradation Features

Each FD001 row is one cycle of one engine: the cycle number, three operating settings and 21 sensor readings. A single reading is noisy and says little about how far an engine has degraded, so we add features that summarise each sensor's recent history. We use the 14 informative sensors selected by the exploratory analysis, which drops the six constant sensors and the near binary s_6. All features are computed by `make_features` in `src/rul/features.py`, which returns one row per input row with the original index. Every window is computed within a single engine (grouped by `unit`

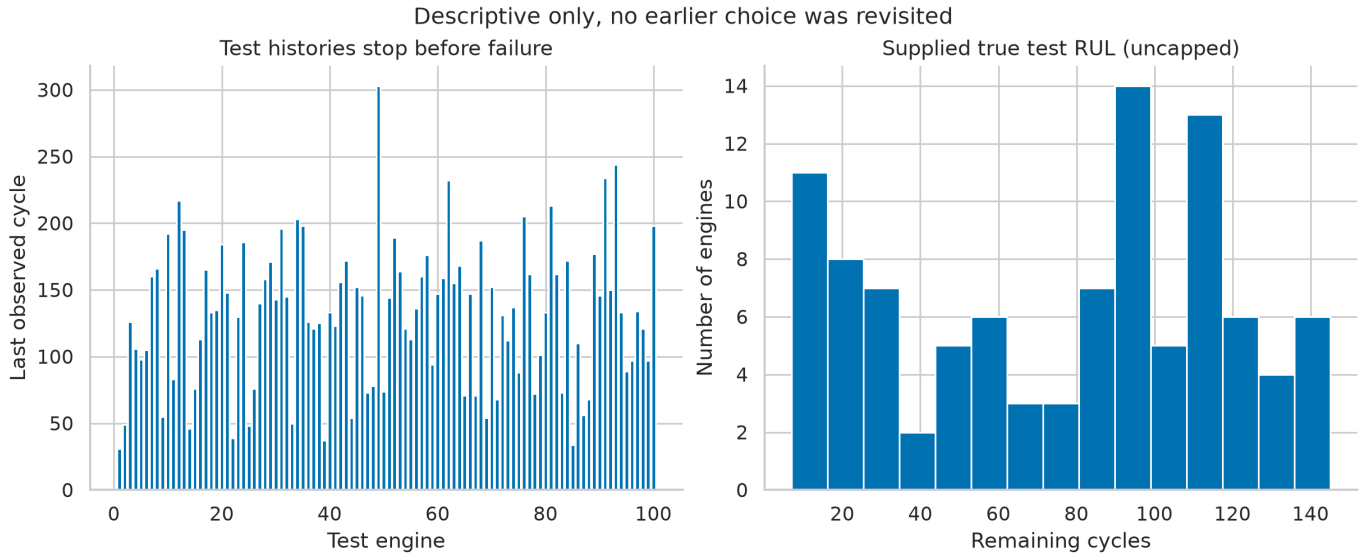


Fig. 6. Final test diagnostics: last observed cycle per engine and true RUL distribution. Descriptive only, no earlier choice was revisited.

and ordered by cycle), so no feature ever mixes readings from two engines.

A. Cycle

The cycle number t is kept unchanged as a feature. It is the time since the engine entered service in the dataset, so on its own it is a weak prior on wear. It is not scaled, since tree models are invariant to monotone rescaling and downstream code uses the raw cycle.

B. Operating regime normalisation

Sensor levels depend on the flight condition (altitude, Mach number and throttle angle) as much as on wear. We standardise the three settings and cluster them with k-means (seed 42) into K operating regimes. For each regime r and sensor j we store the mean μ_{rj} and standard deviation σ_{rj} over the training rows in that regime, and normalise

$$z_{t,j} = \frac{x_{t,j} - \mu_{r(t),j}}{\sigma_{r(t),j}}, \tag{3}$$

where $r(t)$ is the regime nearest to the row's settings. FD001 runs in one regime ($K = 1$), so this is an ordinary z-score; with $K = 6$ the same code removes the regime offsets of FD002 and FD004, leaving variation due to wear. If a sensor that is constant in a regime is passed in (such as s_1 or s_16 in FD001), its σ is 0 up to floating point noise; it is only centred, so its features are zero rather than amplified noise.

The cluster centres and (μ, σ) are estimated on training engines only and stored in a scaler object that is reused, unchanged, on validation and test engines. If they were re-estimated on the validation engines, the validation features would be scaled by statistics that include those engines' own failure trajectories, and the cross validated error would be optimistic. In cross validation the scaler is refitted in every fold on that fold's training engines.

C. Rolling mean and standard deviation

For window length $w \in \{5, 10, 20\}$ and the set W_t of the last $\min(w, t)$ cycles of the same engine, with $n = |W_t|$,

$$\begin{aligned} m_{t,j}^{(w)} &= \frac{1}{n} \sum_{\tau \in W_t} z_{\tau,j}, \\ \sigma_{t,j}^{(w)} &= \sqrt{\frac{1}{n-1} \sum_{\tau \in W_t} \left(z_{\tau,j} - m_{t,j}^{(w)}\right)^2}. \end{aligned} \tag{4}$$

The mean filters measurement noise and estimates the current level of degradation; the standard deviation measures how erratic the sensor has become, which tends to grow as components wear.

a) Early cycles: The first $w-1$ cycles of an engine have fewer than w past readings. Rather than drop these rows or leave missing values, the window is truncated to the cycles seen so far (`min_periods=1`). At cycle 1 the mean equals the reading and the standard deviation, undefined for one value, is set to 0. These features are therefore noisier for the first $w-1$ cycles of each engine. Those cycles are at least 109 cycles from failure in FD001, where the target is at or near the cap of 125, so this costs little accuracy.

D. Rolling least squares slope

The slope is the ordinary least squares gradient of $z_{\tau,j}$ against cycle τ over the same window. With n points and sums taken over W_t,

$$b_{t,j}^{(w)} = \frac{n \sum \tau \, z_{\tau,j} - \sum \tau \sum z_{\tau,j}}{n \sum \tau^2 - \left(\sum \tau\right)^2}. \tag{5}$$

All five sums are rolling sums, so every slope is computed in closed form in one pass instead of fitting a regression per row. For $n = 1$ the denominator is zero and we define $b = 0$: a single point has no trend.

The slope is the local rate of degradation, in standard deviations per cycle. The rolling mean says how worn an engine is; the slope says how fast it is wearing. In FD001 the sensors drift slowly for most of an engine's life and then accelerate as the fault develops, so a steep slope marks an engine in the final phase of its life even when its sensor levels overlap with those of a healthy engine of a different build. Because the slope uses differences within one engine, it is also insensitive to the engine to engine offsets in initial wear that make raw levels hard to compare. On a window of 2 or 3 points the slope is dominated by noise, which is the main cost of the truncated early windows.

E. Health index

The health index (HI) combines all normalised sensors into one number between 0 and 1. We fit a linear regression on training engines,

$$h_t = 1 - \frac{t-1}{T-1}, \qquad \widehat{\mathrm{HI}}_t = \mathrm{clip}\big(\beta_0 + \boldsymbol{\beta}^\top \mathbf{z}_t,\ 0,\ 1\big), \quad (6)$$

where T is the engine's failure cycle, so the target h falls linearly from 1 at the first cycle to 0 at failure. At prediction time no T is needed: the fitted coefficients map any engine's sensors to an HI. We chose regression over the first principal component because PCA finds the direction of greatest variance, which need not be the direction of wear, and its sign and scale are arbitrary; the regression picks the sensor combination that tracks degradation directly and has a fixed 0 to 1 scale. Its limitation is the linear target: real wear is flat for most of a life and then accelerates, so the HI starts near 0.7 rather than 1 and only falls clearly in the last third of life. Fitted on each cross validation fold's training engines and evaluated on its validation engines, the HI has an out of fold correlation of 0.87 ± 0.02 with the capped RUL (0.86 in sample, so the fit does not overfit). For comparison, the strongest single feature, the 5 cycle rolling mean of s_4, reaches $|r| = 0.82$ in sample.

Fig. 7 shows these features for two training engines: the 20 cycle rolling mean removes most of the cycle to cycle noise, the 20 cycle slope moves away from zero as each engine nears failure, and the HI falls from near 0.8 towards 0 along each life.

F. Ablation

Table II adds the feature families one at a time to a Random Forest, scored with five fold cross validation grouped by engine on every validation cycle. The forest uses default hyperparameters (seed 42), so the comparison reflects the features rather than tuning. The tuned forest of Section VI reaches an RMSE of 14.42 on the full engineered set (Table IV).

Smoothing helps a little: a forest already averages over many noisy rows. The slope gives the largest gain, about 1.6 RMSE and 2.5 late stage RMSE, well beyond the fold to fold standard deviation of 0.5 to 0.8. The HI adds a further small gain within that noise, since it is a linear combination of sensors the forest already sees; its main value is as one interpretable degradation signal.

TABLE II
FEATURE ABLATION, 5 FOLD ENGINE CROSS VALIDATION ON FD001 TRAINING ENGINES (MEAN OVER FOLDS, RUL CAPPED AT 125).

Features	n	RMSE	MAE	Late RMSE
Cycle and raw sensors	15	16.81	11.53	15.54
+ rolling mean and std	99	16.36	11.04	13.86
+ rolling slope	141	14.78	10.04	11.37
+ health index	142	14.74	9.97	11.24

G. Limitation of fixed windows

Every feature here looks back at most 20 cycles, while FD001 engines run for 128 to 362 cycles. A fixed window cannot tell whether a sensor has been drifting for 150 cycles or has just started to move, cannot see when the drift began to accelerate, and must use one window length for engines that degrade at very different rates. Lengthening the window trades this against slower response and longer truncated starts, and the choice of windows (5, 10, 20) is itself a hand-tuned hyperparameter. These are the gaps that motivate Phase 2: sequence models such as LSTMs or temporal convolutions take the whole trajectory as input and learn which part of the history matters, rather than relying on fixed hand-made summaries.

VI. Models and Tuning

Each training row is one cycle of one engine, and the task is to regress its RUL, capped at 125 cycles. We compare two naive baselines with three models of increasing flexibility: Elastic Net, a regularised linear model; Random Forest, an average of deep trees; and XGBoost, a sum of shallow boosted trees. Each model is trained on two feature sets: the raw set (cycle number and the 14 informative sensors, 15 features) and the engineered set of Section V (142 features). All models are built by `build_model` in `src/rul/models.py`. Random Forest and XGBoost are seeded with 42; Elastic Net has no random component, since its coordinate descent updates the coefficients in a fixed cyclic order.

A. Baselines

The mean predictor returns the mean training RUL for every row; it ignores the sensors and sets the error that any useful model must improve on. Ordinary least squares on the raw features is the simplest model that uses the sensors. Every tuned model is expected to beat both. Linear regression is also fitted on the engineered features, as an unregularised reference for Elastic Net on the same inputs.

B. Elastic Net

Elastic Net [13] is linear regression with a penalty that mixes the L1 and L2 norms of the coefficients. In the parametrisation used by scikit-learn, with n rows, features $\mathbf{x}_i$ and targets y_i, it solves

$$\min_{\beta_0, \boldsymbol{\beta}} \frac{1}{2n} \sum_{i=1}^{n} \big(y_i - \beta_0 - \boldsymbol{\beta}^\top \mathbf{x}_i\big)^2 + \alpha\rho \,\|\boldsymbol{\beta}\|_1 + \frac{\alpha(1-\rho)}{2} \,\|\boldsymbol{\beta}\|_2^2, \quad (7)$$

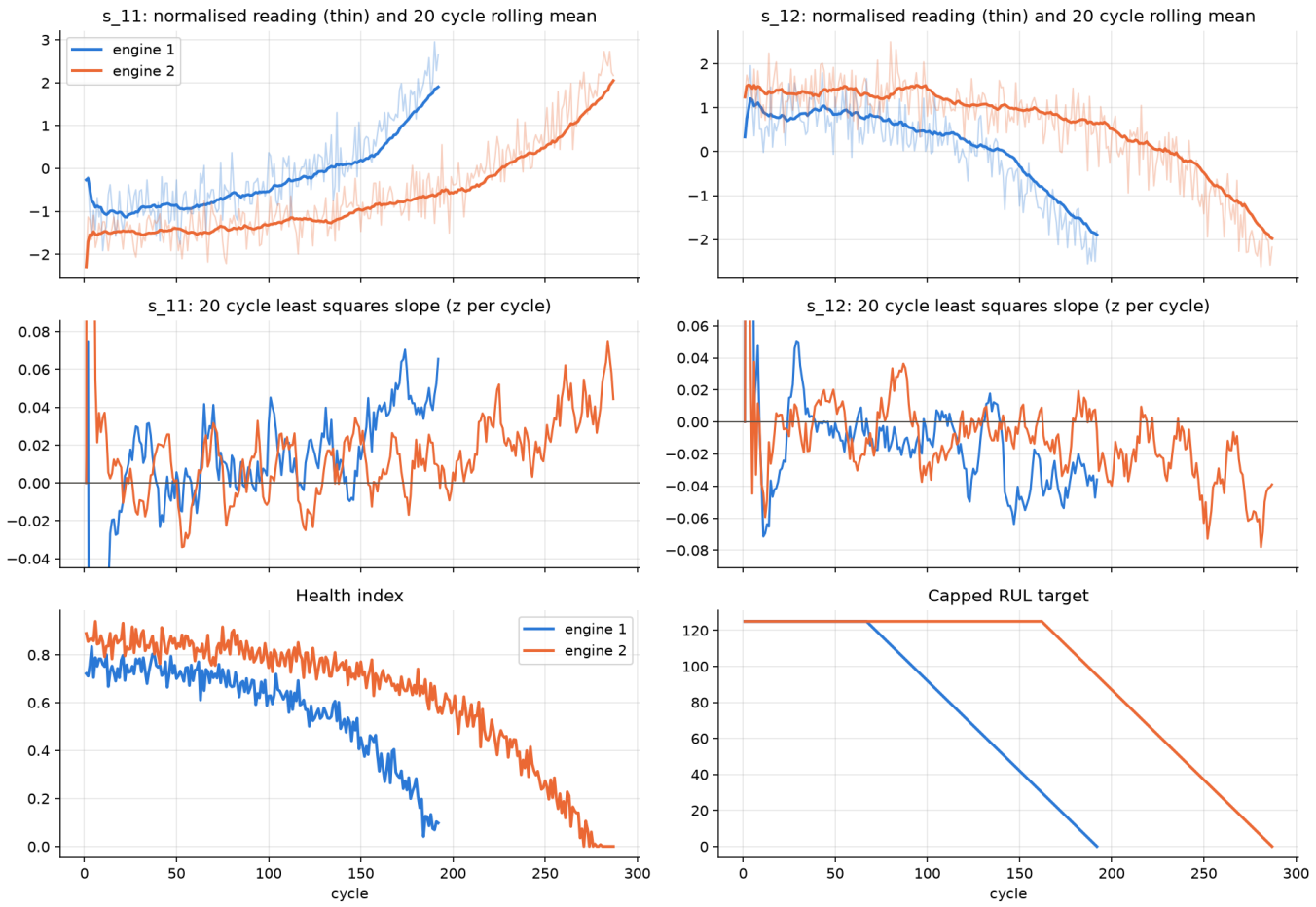


Fig. 7. Engineered features for training engines 1 and 2. Top: regime normalised readings of `s_11` (rises with wear) and `s_12` (falls with wear), thin, with their 20 cycle rolling means. Middle: 20 cycle least squares slopes; the axis is scaled to cycles 20 onwards, since slopes over the first few partial windows are large and meaningless. Bottom: health index and capped RUL target.

where $\alpha \geq 0$ sets the overall strength of the penalty and $\rho \in [0, 1]$ (`l1_ratio`) its L1 share. The L1 term can set coefficients exactly to zero, so it selects features. On its own (the Lasso, $\rho = 1$) it tends to keep one feature from a group of correlated features and drop the others more or less arbitrarily, and the choice can change between samples. The L2 term shrinks coefficients smoothly towards zero, spreads weight across correlated features and makes the solution stable. Mixing the two gives sparse models that remain stable when features are strongly correlated, as the rolling statistics of one sensor over three windows are.

The penalty depends on the scale of each coefficient, so features are standardised first. The scaler and the regression form one scikit-learn `Pipeline`, so in cross validation the scaler is fitted on each fold's training engines only. Elastic Net is linear in its inputs, so it has high bias and low variance. On raw sensors it cannot follow the flat-then-accelerating shape of degradation; it can represent nonlinear behaviour only through features that already encode it, such as rolling slopes and the health index. The penalty lowers variance further at the cost of more bias.

C. Random Forest

A regression tree splits the feature space into boxes and predicts the mean target in each box. A deep tree has low bias but high variance: a small change in the training data can change its splits completely. Random Forest [14] trains B trees, each on a bootstrap sample of the rows (bagging), considers only a random subset of the features at each split, and averages the predictions. If each tree has variance σ^2 and two trees have correlation ρ, the variance of the average is

$$\rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2. \tag{8}$$

Adding trees removes the second term, and sampling features at each split lowers ρ, so the forest keeps the low bias of deep trees while reducing their variance. Trees are trained independently, and as trees are added the forest's generalisation error converges [14]: adding trees does not increase overfitting; the error levels off, so more trees only cost time.

D. Gradient boosting and XGBoost

Boosting builds an additive model one tree at a time, each tree correcting the errors of the trees before it [15]:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, f_m(\mathbf{x}), \tag{9}$$

where f_m is a shallow tree fitted to the negative gradient of the loss at F_{m-1} and η is the learning rate. For squared error the negative gradient is the residual $y_i - F_{m-1}(\mathbf{x}_i)$, so each tree is fitted to what the current model still gets wrong. Where bagging averages independent low-bias trees to cut variance, boosting adds dependent high-bias trees to cut bias. Each added tree fits the training data more closely, so the number of rounds, the learning rate and the tree depth must be limited to control variance.

XGBoost [16] adds regularisation to the boosting objective. Tree m minimises

$$\sum_{i=1}^{n} \ell\big(y_i, F_{m-1}(\mathbf{x}_i) + f_m(\mathbf{x}_i)\big) + \gamma T + \frac{\lambda}{2}\|\mathbf{w}\|_2^2, \tag{10}$$

where T is the number of leaves and $\mathbf{w}$ the leaf values. The γ term charges for each extra leaf, and the λ term (`reg_lambda`) shrinks leaf values, so a leaf supported by few rows moves the prediction less. XGBoost uses a second order Taylor expansion of the loss to choose splits and leaf values, and can also subsample rows (`subsample`) and features (`colsample_bytree`) for each tree, which adds bagging-like randomness.

E. Tuning setup

All tuning uses the five engine-grouped folds of Section IV, so every engine is validated exactly once and no engine appears on both sides of a split. The engineered features depend on statistics fitted to training data; they are rebuilt in every fold from that fold's training engines and then applied to its validation engines. Each model is tuned separately on each feature set, and the configuration with the lowest mean validation RMSE is selected. Elastic Net is cheap, so its full grid is searched, solved by coordinate descent on a precomputed Gram matrix with up to 100,000 iterations so that every configuration converges; for Random Forest and XGBoost a fixed number of configurations is drawn at random from the grid with seed 42. Table III lists the search spaces and the selected values. The test set is not used.

F. Cross validation results

Table IV reports each selected configuration. RMSE and MAE are the mean and standard deviation over the five folds, computed on every held-out cycle against the capped target. Late RMSE restricts the error to cycles with true RUL of at most 50, and the last column to cycles with true RUL of at most 125, where capped and true labels agree.

These are model-selection scores. The same folds are used to choose each configuration and to report its score, so the scores of the selected configurations are optimistic: among several configurations, the one that did best on these folds is partly the one that was luckiest on them. Grouping by engine prevents leakage between engines, and a small search averaged over five folds limits the optimism, but neither removes this selection bias. Nested cross validation would remove it; here the independent estimate comes instead from the test set, which is scored once with the frozen configurations (Section IV).

TABLE III
SEARCH SPACES. SELECTED VALUES ARE IN PARENTHESES, GIVEN AS RAW / ENGINEERED WHERE THE TWO FEATURE SETS DIFFER. RANDOM FOREST AND XGBOOST SAMPLE 6 AND 8 CONFIGURATIONS FROM THEIR GRIDS; ELASTIC NET SEARCHES ALL 20.

Model	Hyperparameter	Values (selected)
Elastic Net	`alpha`	0.001, 0.01, 0.1, 0.3, 1.0 (0.1 / 0.001)
	`l1_ratio`	0.2, 0.5, 0.8, 1.0 (1.0 / 0.8)
Random Forest	`n_estimators`	100, 200 (200 / 100)
	`max_depth`	8, 12, none (12 / none)
	`min_samples_leaf`	1, 5, 10 (1 / 5)
	`max_features`	0.3, 0.5, 1.0 (0.3 / 0.5)
XGBoost	`n_estimators`	200, 400 (400)
	`learning_rate`	0.03, 0.05, 0.1 (0.03 / 0.05)
	`max_depth`	3, 4, 6 (4)
	`subsample`	0.8, 1.0 (1.0)
	`colsample_bytree`	0.5, 0.8, 1.0 (0.5 / 0.8)
	`reg_lambda`	1, 5 (5)

Engineered features improve every model, by 2.0 to 3.1 RMSE, well beyond the fold standard deviations. The tree ensembles beat the linear models by a wide margin on both feature sets. Degradation in the sensors is nonlinear and accelerates near failure. A linear model can follow this only to the extent that its features already encode it: the rolling statistics, slopes and health index supply part of that structure, which is why linear regression improves from 20.12 to 17.07 with them. Trees learn the remaining nonlinearity directly, piece by piece. XGBoost on engineered features has the lowest error on every metric, though Random Forest is within one fold standard deviation on overall RMSE; XGBoost's clearer advantage is on late-life cycles (9.88 against 10.98).

All tuned models beat the mean baseline. Against the linear regression baseline on raw features, both tree ensembles are better on both feature sets and Elastic Net is better on engineered features (17.06 against 20.12). On each feature set, Elastic Net ties with unregularised linear regression on the same inputs: on raw features its RMSE is 0.0006 lower and its MAE 0.0015 higher, and on engineered features its RMSE is 0.003 lower and its MAE 0.0003 higher, all far inside the fold standard deviations. With at most 142 features and over 16,000 training rows per fold, the linear model is limited by bias rather than variance, so the penalty has almost nothing to remove. On engineered features the error rises steadily with α (17.75 at $\alpha = 0.1$ and 19.39 at $\alpha = 1$, both pure L1), and the search selects the smallest value in the grid, $\alpha = 0.001$; smaller values only bring Elastic Net closer to least squares, whose score is the linear regression row. The penalty does not improve predictive accuracy here; Elastic Net's gain over the raw baseline comes from the engineered features, not from

TABLE IV

FIVE FOLD ENGINE CROSS VALIDATION ON FD001 TRAINING ENGINES. ROWS ABOVE THE LINE ARE TRAINED ON RUL CAPPED AT 125 AND SCORED AGAINST IT. THE TWO ROWS BELOW THE LINE RETRAIN THE BEST CONFIGURATION WITH AND WITHOUT THE CAP AND SCORE BOTH AGAINST THE TRUE RUL, SO THEIR RMSE AND MAE ARE COMPARABLE WITH EACH OTHER BUT NOT WITH THE ROWS ABOVE; THE LAST COLUMN IS COMPARABLE ACROSS ALL ROWS.

Model	Features	Target	RMSE	MAE	Late RMSE	RMSE, RUL ≤ 125
Mean baseline	–	capped	41.67 ± 0.00	36.97 ± 0.00	63.56	43.76
Linear regression	raw	capped	20.12 ± 1.54	16.37 ± 1.29	21.71	20.79
Elastic Net	raw	capped	20.12 ± 1.54	16.37 ± 1.29	21.72	20.78
Random Forest	raw	capped	16.43 ± 0.77	11.47 ± 0.71	15.40	18.29
XGBoost	raw	capped	16.42 ± 0.87	11.38 ± 0.70	15.23	18.28
Linear regression	engineered	capped	17.07 ± 0.78	13.72 ± 0.63	16.89	17.65
Elastic Net	engineered	capped	17.06 ± 0.80	13.72 ± 0.64	16.86	17.63
Random Forest	engineered	capped	14.42 ± 0.48	9.87 ± 0.56	10.98	15.76
XGBoost	engineered	capped	$\mathbf{13.99} \pm 0.46$	$\mathbf{9.81} \pm 0.42$	**9.88**	**15.09**
XGBoost	engineered	capped	47.47 ± 2.21	30.38 ± 0.77	9.88	15.09
XGBoost	engineered	uncapped	38.26 ± 1.41	26.16 ± 1.65	11.20	25.99

regularisation.

G. Effect of the RUL cap

Capping assumes that degradation is not visible early in life, so a model should not be asked to tell 300 remaining cycles from 200. To test this, the best configuration was retrained on the uncapped RUL with the same folds, and both versions were scored against the true RUL (the last two rows of Table IV). The answer depends on which cycles are scored. On cycles with true RUL of at most 125, the capped model is far more accurate (RMSE 15.09 against 25.99), and it is also better on late-life cycles (9.88 against 11.20). Over all cycles the uncapped model is better (38.26 against 47.47). The capped model never sees a training target above 125, so its predictions for young engines stay close to 125 while their true RUL is several hundred cycles. Capping changes the training target rather than bounding the predictions: it concentrates the model on the part of life where degradation is visible, trading accuracy on early-life cycles, where predictions have little practical use, for accuracy close to failure, where maintenance decisions are made. We keep the cap for all models.

H. Selected model

XGBoost on engineered features, trained on RUL capped at 125, is carried forward to the test set with the hyperparameters in Table III. The selected configuration of every model on both feature sets is saved in `results/best_params.json`, together with the sensor list and window lengths needed to rebuild the features, so test evaluation uses exactly the configurations chosen here.

I. Explainability

We examine what each selected model relies on, using training engines only (`notebooks/05_explain.ipynb`). The 142 engineered columns are strongly correlated, since each sensor contributes its raw reading and a rolling mean, standard deviation and slope over three windows. All results are therefore also summed into 16 groups: one per sensor, plus `cycle` and `health_index`.

Impurity importance is the training loss reduction from the splits that use a feature: the mean decrease in squared error for Random Forest and `total_gain` for XGBoost (XGBoost's default `gain` is an average per split and cannot be summed over a group). *Permutation importance* [14] is the rise in RMSE when a feature's values are shuffled across rows. We compute it on held-out engines: in each of the five engine folds the model is refitted on the training engines and shuffled on the validation engines, five times per fold. All columns of a group are shuffled with the same row order. Shuffling one column at a time would understate a sensor whose information is duplicated in its other columns, because the model can recover it from the unshuffled copies.

Both tree ensembles depend most on the health index (Fig. 8). Shuffling it on held-out engines raises RMSE by 10.95 cycles for XGBoost and 7.36 for Random Forest, and shuffling `cycle` by 4.84 and 5.22; no other group adds more than 1.62. Below these, the two measures disagree. `s_4` receives 26% of Random Forest's impurity reduction but costs only 1.62 cycles when shuffled, because its information is also carried by `s_11`, the other degrading sensors and the health index. Impurity importance credits whichever correlated feature a split happened to use, while permutation importance measures what the model loses when that information is gone. `cycle` shows the reverse: 8 to 9% of impurity reduction but 20 to 26% of permutation importance, since no other input carries the engine's age. Impurity importance also describes the training rows, including splits that fit noise, while held-out permutation importance measures dependence on unseen engines.

A SHAP value [17] is the contribution of one feature to one prediction relative to the model's average prediction. For each row the contributions add up exactly: the base value (the mean prediction) plus the row's SHAP values equals the prediction. Each value is the feature's Shapley value, its average marginal effect over all orders in which features can be added. `shap.TreeExplainer` computes these exactly for tree ensembles [18]. We explain the XGBoost model fitted on all training engines on 2,000 seeded training rows; the largest additivity gap is 1.4×10^{-4} cycles, around a base value of

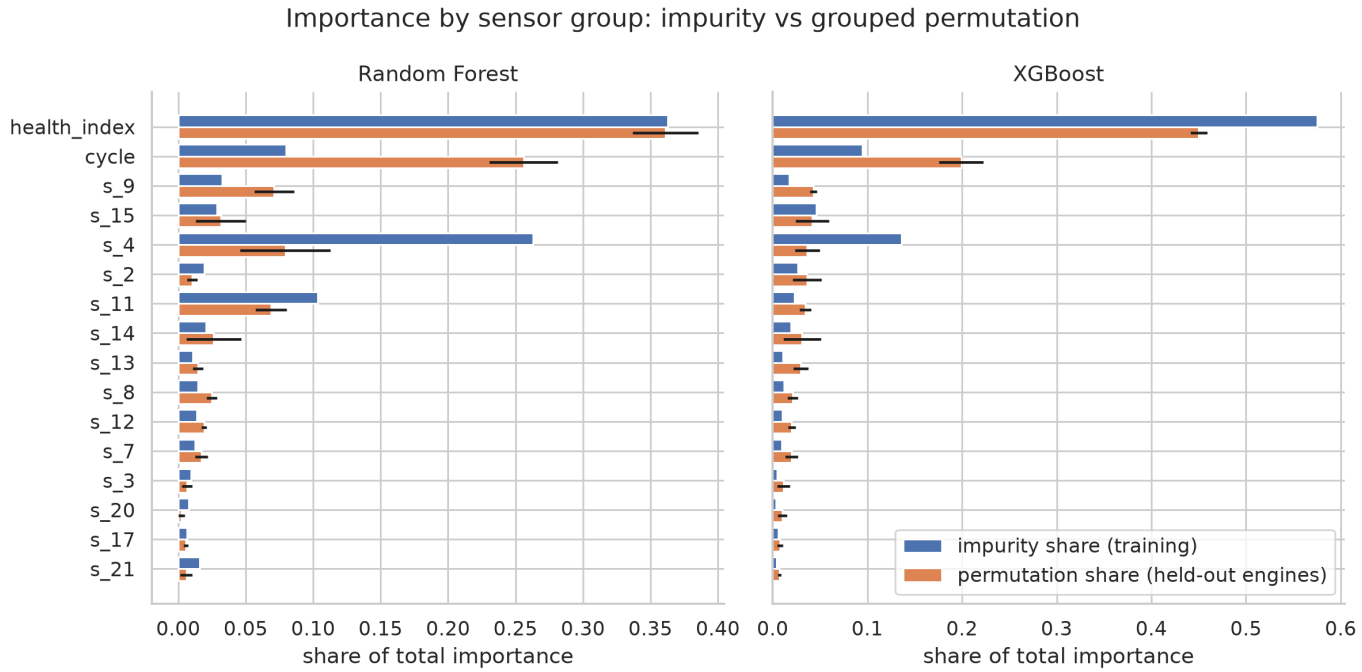


Fig. 8. Importance of each sensor group as a share of the total: impurity importance on training rows and grouped permutation importance on held-out engines (mean and fold standard deviation over five engine folds).

86.83 (Fig. 9). A low health index lowers the prediction by up to 40 cycles, and late cycles lower it. The sensor effects are smaller and follow the degradation trends: high recent means and positive slopes of `s_2`, `s_4`, `s_11` and `s_15` lower the predicted RUL, and negative slopes of `s_7` and `s_12` lower it. Every slope among the top 20 columns uses the 20-cycle window.

Elastic Net standardises its inputs, so its coefficients are comparable in size. At the selected penalty ($\alpha = 0.001$) no coefficient is zero, and among the largest after the health index are pairs of rolling means of one sensor over two windows with opposite signs, such as `s_11_mean_20` $+22.5$ with `s_11_mean_10` -20.1, or `s_12_mean_20` -21.1 with `s_12_mean_10` $+16.3$. Each pair acts as a trend estimate, and neither sign means anything alone. Strengthening a pure L1 penalty zeroes 38 coefficients at $\alpha = 0.1$ and 116 at $\alpha = 1$. By $\alpha = 1$ every single-cycle sensor reading is gone, as a noisier copy of its 5-cycle mean, and only `cycle` remains of the raw columns, but every sensor keeps at least one column. With correlated inputs L1 keeps one column of a correlated set somewhat arbitrarily, so a zero means "redundant given the others", not "unrelated to RUL". These sparser models are also less accurate (17.75 and 19.39 against 17.06).

The models do rely on the sensors the exploratory analysis found degrading, but mostly through the health index, a linear combination of all 14 sensors whose largest weights are on `s_11`, `s_9`, `s_12`, `s_4` and `s_7`. Among individual sensors, `s_4`, `s_15` and `s_2`, all of which rise towards failure, lead XGBoost's SHAP ranking. `s_7` and `s_12` correlate strongly with RUL but rank low, since their information is largely duplicated by `s_4`, `s_11` and the health index. Reliance on `cycle`, second under SHAP and permutation importance, means the models partly predict from age. This works on FD001, where every engine starts in normal condition and training lifetimes span 128 to 362 cycles, but it would transfer less well to fleets with different lifetimes.

VII. Test Set Results

A. Protocol

Every hyperparameter and the choice of selected model were fixed by cross validation (Section VI) before the test set was opened. Each configuration is refitted on all 100 training engines with RUL capped at 125 cycles, and the feature statistics (regime normalisation and the health index) are fitted on the training engines and applied unchanged to the test engines. Features are computed over each test engine's full observed history, and the model predicts once per engine at its last observed cycle (Section IV). Predictions are scored against the true, uncapped RUL. The overall and stage metrics are taken from `results/test_metrics.csv` and the published comparison from `results/test_published_comparison.csv`; the bootstrap intervals, the error by RUL bucket and the per-engine terms of S are computed in `notebooks/04_results.ipynb`.

B. Overall results

Table V gives the results. The selected XGBoost model reaches an RMSE of 15.36 cycles, an MAE of 11.26 and a score of 389. In the late stage, where maintenance is

TABLE V

TEST RESULTS ON THE 100 FD001 TEST ENGINES, SCORED AGAINST THE TRUE RUL. LATE STAGE IS TRUE RUL ≤ 50 (33 ENGINES), EARLY STAGE IS TRUE RUL > 50 (67 ENGINES). XGBOOST ON ENGINEERED FEATURES WAS SELECTED BY CROSS VALIDATION BEFORE THE TEST SET WAS USED.

Model	Features	RMSE	MAE	Score S	Early RMSE	Late RMSE	Late MAE
Mean baseline	–	43.07	35.90	33629	27.30	64.09	62.68
Linear regression	raw	22.54	17.73	1291	21.93	23.72	19.83
Elastic Net	raw	22.57	17.77	1296	21.93	23.82	19.95
Random Forest	raw	17.77	13.21	534	19.42	13.83	8.90
XGBoost	raw	17.58	13.01	539	19.39	13.17	8.22
Linear regression	engineered	18.23	14.10	519	19.69	14.85	12.68
Elastic Net	engineered	18.27	14.11	522	19.75	14.83	12.64
Random Forest	engineered	15.23	11.33	344	17.88	7.34	5.06
XGBoost (selected)	engineered	15.36	11.26	389	18.11	7.03	5.25

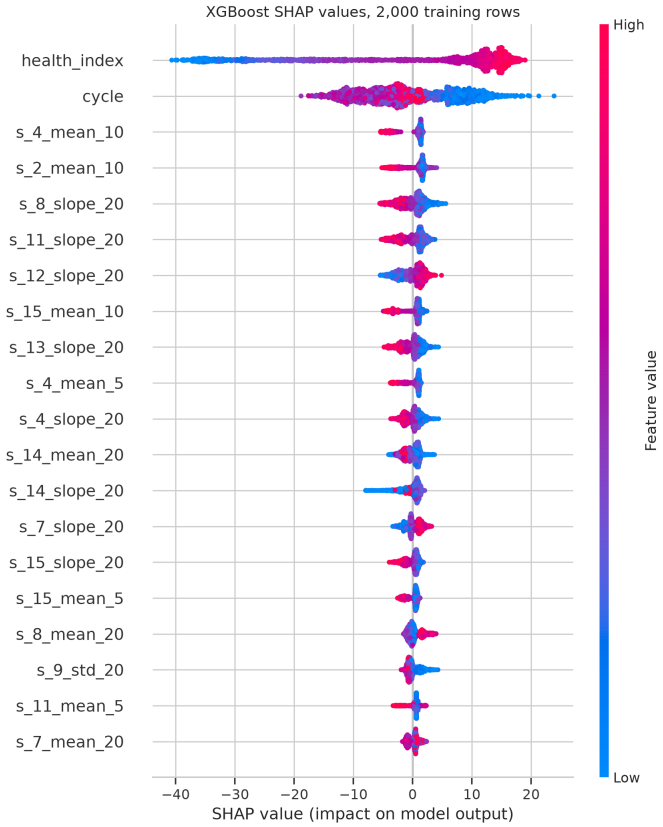


Fig. 9. SHAP values of the selected XGBoost model on 2,000 training rows, for the 20 columns with the largest mean absolute SHAP value. Colour is the feature's value.

scheduled, its RMSE is 7.03 cycles. Random Forest is slightly better on the test set (RMSE 15.23, score 344), although cross validation had ranked XGBoost first (13.99 against 14.42). We keep the selection, because changing it after seeing test results would make the test set a second validation set and bias the reported error downwards.

With only 100 test engines, a gap this small may be sampling noise. A paired bootstrap, resampling the 100 engines 10,000 times and scoring both models on the same resample, gives a 95% interval of -1.3 to $+1.1$ cycles for the RMSE of Random Forest minus XGBoost, so the two ensembles cannot be separated. The intervals for XGBoost minus Elastic Net (-4.8 to -1.0) and for XGBoost on engineered minus raw features (-4.4 to -0.2) exclude zero. The two conclusions of cross validation therefore hold on unseen engines: tree ensembles beat the linear models, and the engineered features improve every model, most of all in the late stage, where XGBoost's RMSE falls from 13.17 to 7.03.

C. Where the errors are

Fig. 10 and Fig. 11 show how the error depends on the true RUL. For both tree ensembles the error is smallest close to failure, with RMSE of about 7 cycles for true RUL of at most 25 and about 8 between 26 and 50. It rises to between 15.7 and 17.5 cycles from 51 to 125. Elastic Net is close to the ensembles far from failure but about twice as inaccurate in the late stage, which is where the trees gain most. Between 51 and 75 cycles all three models predict too late (median error $+9$ to $+14$ cycles), around the point where degradation first becomes visible in the sensors. Eleven test engines have a true RUL above the training cap (up to 145). No model on engineered features predicts above 130, because no training target was larger than 125, so these engines are underestimated by 23 to 26 cycles on average, about 10 of them by construction. Underestimating brings maintenance forward, which is the cheaper error.

The residual distributions (Fig. 12) are centred near zero, with mean errors between -1.0 and $+0.1$ cycles. The difference in score between the two ensembles does not come from late predictions, which add almost the same to S (180.2 and 179.5). Most of the gap between 389 and 344 comes from engine 45 (true RUL 114), predicted at 54 by XGBoost and 64 by Random Forest, which adds 96.8 and 47.4 to S. Without it XGBoost would have the lower score.

D. Comparison with published results

Published FD001 results typically train on RUL capped at 125 and score one prediction per test engine, as we do, but many also clip the test labels at the cap. For a like for like comparison we rescore our models against the test RUL clipped at 125. Table VI lists every FD001 result with both RMSE and score collected by Ragab et al. [4], except an SVM baseline (RMSE 40.72). By RMSE our Random Forest (14.02) and XGBoost (14.08) rank fifth and sixth. They improve on the published Random Forest (17.91) and gradient boosting

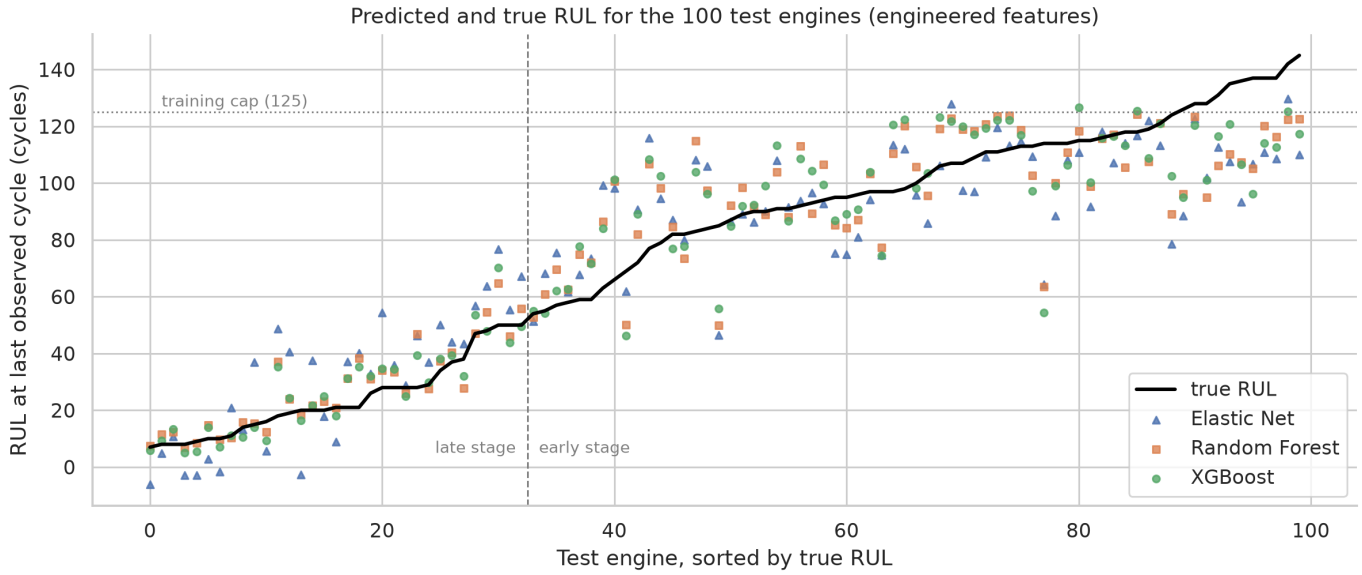


Fig. 10. Predicted and true RUL at the last observed cycle of each test engine, sorted by true RUL. The dashed line separates the late stage (left) from the early stage; the dotted line is the training cap.

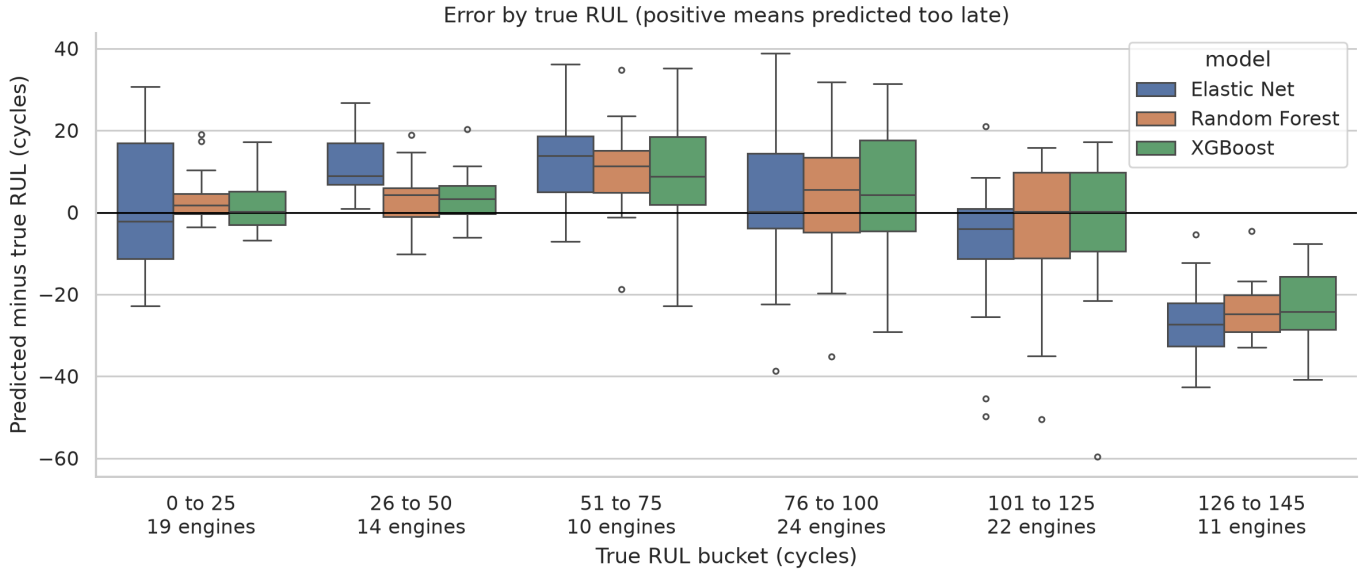


Fig. 11. Prediction error (predicted minus true RUL) by true RUL bucket for the three tuned models on engineered features. Positive errors are late predictions.

(15.67) of Zhang et al. [6] and on six deep models, from CNN-LSTM (14.40) to the 2D CNN of Babu et al. [3] (18.45). Our own raw feature models score 16.58 and 16.37 against the same labels, so in our setting most of the improvement comes from the engineered features rather than the learner.

Four published deep models are more accurate: the 1D CNN of Li et al. [9] (12.61), the attention model of Ragab et al. [4] (12.63), HDNN (13.02) and BLCNN (13.18), between 0.8 and 1.4 cycles of RMSE ahead of our Random Forest. On the score S our position is weaker: six published models score lower than our Random Forest (310), and eight lower than XGBoost (347), including BiLSTM-ED and CNN-LSTM, whose RMSE is higher than ours but whose predictions are less often badly late. The models ahead of ours learn features from windows of the raw sensor sequence; phase 2 of this project tests sequence models of this kind. Published protocols differ in details that are not always reported, such as window length, so differences of a fraction of a cycle should not be over read.

VIII. Deep Learning Setup

A. Sensor windows

The sequence models read a window of the last w cycles of the 14 useful sensors and predict the RUL at the window's final cycle. Windows are built separately for each engine, so no

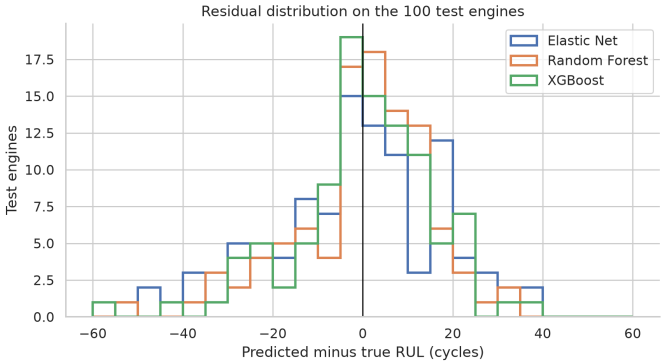


Fig. 12. Distribution of test errors for the three tuned models on engineered features.

TABLE VI
FD001 TEST RESULTS AGAINST LABELS CLIPPED AT 125; OURS USE THE ENGINEERED FEATURES. PUBLISHED VALUES AS COLLECTED IN [4], TABLE III, WHICH CITES THE ORIGINAL PAPERS FOR ROWS WITHOUT A REFERENCE HERE.

Method	RMSE	Score S
1D CNN [9]	12.61	274
ATS2S [4]	12.63	243
HDNN	13.02	245
BLCNN	13.18	302
Random Forest (ours)	14.02	310
XGBoost (ours)	14.08	347
CNN-LSTM	14.40	290
BiLSTM-ED	14.74	273
LSTMBS	14.89	481
MODBNE [6]	15.04	334
Gradient boosting [6]	15.67	474
Deep LSTM [7]	16.14	338
Elastic Net (ours)	17.01	469
Random Forest [6]	17.91	480
2D CNN [3]	18.45	1287

window ever contains cycles from two engines. We build one window ending at every cycle. This gives the sequence models the same scored rows and folds as the Phase 1 models. One difference remains: the sequence models train on 64 engines per fold, because 16 of the fold's 80 training engines are held out for early stopping, while the Phase 1 models train on all 80. The default length is $w = 30$: the shortest FD001 test engine has 31 observed cycles, so every test engine is scored on real history, while the shortest training engine (128 cycles) is far longer than the window.

B. Padding

The first $w - 1$ cycles of every engine have fewer than w cycles of history. Rather than drop them, which would remove exactly the healthy early cycles, we left pad each window with copies of that engine's first cycle. A healthy engine's sensors are nearly constant, so repeating the first reading adds no false degradation trend, and the padding never borrows readings from another engine.

C. Normalisation

Each sensor is standardised with the mean and standard deviation of the training engines only. FD001 has a single operating regime, so standardising by regime and standardising globally coincide. In cross validation the scaler is refitted inside every fold on that fold's training engines, so no statistic of a scored engine reaches the model.

D. Training and early stopping

All sequence models share one training loop: Adam on mean squared error, mini batches of 256 windows, gradient norms clipped at 1, and seed 42 for weights and batch order. The loop multiplies every network output by 125, the RUL cap, so a freshly initialised network, whose outputs start near zero, only has to learn values near 1 instead of values up to 125. Training stops after a fixed number of epochs without improvement in validation loss, and the weights from the best epoch are restored. Early stopping chooses a hyperparameter, the number of epochs, by looking at validation error. If it watched the same engines that are later scored, the reported error would be optimistic. We therefore nest the split: within each of the five `GroupKFold` folds, 20% of the training engines are held out for early stopping, and the fold's held out engines are used only for scoring. As in Phase 1, every cycle of the scored engines is compared with RUL capped at 125.

IX. SEQUENCE MODELS

Three networks read the sensor windows of Section VIII directly: an LSTM, a temporal convolutional network (TCN) and a Transformer encoder. Each maps a window of 30 cycles of the 14 useful sensors to one number, which the training loop multiplies by 125. None of them sees the cycle number or the engineered features of Phase 1, so any skill they show comes from the raw sensor history alone.

A. Architectures

LSTM. An LSTM [8], in the now standard form with the forget gate of Gers et al. [19], reads the window one cycle at a time. At cycle t it combines the reading x_t with the previous hidden state h_{t-1} through three gates and a candidate update,

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad g_t = \tanh(W_g[h_{t-1}, x_t] + b_g),$$
$$c_t = f_t \odot c_{t-1} + i_t \odot g_t, \quad h_t = o_t \odot \tanh(c_t).$$

The cell state c_t is updated by addition, so the gradient can flow back across many cycles without vanishing. We stack one or two layers and pass the hidden state after the last cycle through dropout to a linear output.

Temporal CNN. Following Bai et al. [20], the TCN is a stack of residual blocks. Each block applies two 1D convolutions with kernel size $k = 3$, each followed by ReLU and dropout, and adds the block's input back through a skip connection (a 1×1 convolution when the channel count changes). The convolutions are causal: the input is padded on the left only, so the output at cycle t depends on cycles up to t and never on later ones. Block j uses dilation 2^{j-1},

so the gap between the cycles one filter reads doubles from block to block. With L blocks the last output sees

$$R = 1 + 2(k-1)(2^L - 1)$$

cycles: 29 for three blocks and 61 for four. Three blocks therefore cover 29 of the 30 cycles in the window and four cover all of it. The output at the last cycle feeds a linear layer.

Transformer encoder. Each cycle's 14 readings are projected to a d-dimensional vector and a sinusoidal positional encoding [21] is added, followed by dropout. One or two encoder layers follow, each with four attention heads, a feed-forward layer of width $4d$ and dropout. The 30 outputs are averaged and passed to a linear layer. Self-attention and averaging both ignore the order of their inputs, so without the positional encoding the model would give the same prediction for a window and for the same window shuffled. It could not tell a rising sensor from a falling one. The positional encoding gives each cycle a distinct code, which the attention can use to recover order.

B. Regularisation

Every network is regularised in four ways. *Dropout* [22] sets a random fraction of activations to zero at each training step, so no unit can rely on one particular partner, and turns off at prediction time. *Weight decay* adds 10^{-4} times the weights to every gradient step, an L2 penalty that keeps the weights small. *Gradient clipping* rescales any gradient whose norm exceeds 1, so one batch with an unusually large gradient, which recurrent networks occasionally produce, cannot swamp the running averages that Adam uses to scale its steps. It does not limit the capacity of the model. *Early stopping* ends training once the loss on the held-out early-stopping engines has not improved for 10 epochs and restores the best weights, so the number of epochs is chosen on engines the model was not trained on.

C. Tuning

Each network is tuned with the five engine folds of Phase 1. For each one we sample eight configurations, with seed 42, from a grid of 16 over width, depth, dropout (0.1 or 0.3) and learning rate, and keep the configuration with the lowest mean RMSE. Table VII lists the grids and the selected values. Adam, weight decay 10^{-4}, batches of 256 and patience 10 are fixed for every run.

D. Cross validation results

Table VIII puts the selected networks next to the Phase 1 models. Every row uses the same folds, the same scored cycles and RUL capped at 125, but the training data differ: the networks train on 64 of each fold's 80 training engines, because the other 16 are held out for early stopping, while the Phase 1 models train on all 80. The networks therefore learn from a fifth fewer engines but use those 16 to choose when to stop, so the comparison is close but not exactly like for like. The Transformer has the lowest error of any model in either phase, 13.28 against 13.99 for XGBoost on engineered features. The LSTM reaches 14.02 with 6,177 parameters, the fewest of the three networks. The TCN is the weakest network at 15.91.

TABLE VII
SEARCH SPACES OF THE SEQUENCE MODELS, EIGHT OF 16 CONFIGURATIONS SAMPLED PER NETWORK. SELECTED VALUES ARE IN PARENTHESES.

Network	Hyperparameter	Values (selected)
LSTM	hidden size	32, 64 (32)
	layers	1, 2 (1)
	dropout	0.1, 0.3 (0.3)
	learning rate	0.001, 0.003 (0.003)
TCN	channels	32, 64 (64)
	blocks	3, 4 (4)
	dropout	0.1, 0.3 (0.1)
	learning rate	0.001, 0.003 (0.003)
Transformer	d	32, 64 (64)
	layers	1, 2 (1)
	dropout	0.1, 0.3 (0.1)
	learning rate	0.0003, 0.001 (0.001)

TABLE VIII
FIVE FOLD ENGINE CROSS VALIDATION, RUL CAPPED AT 125. LATE RMSE COVERS CYCLES WITH TRUE RUL ≤ 50. PHASE 1 ROWS ARE FROM TABLE IV; THOSE MODELS TRAIN ON 80 ENGINES PER FOLD, THE NETWORKS ON 64.

Model (inputs)	Params	RMSE	MAE	Late RMSE
Transformer (windows)	51,009	13.28 ± 0.34	9.54	7.98
LSTM (windows)	6,177	14.02 ± 0.43	10.00	8.84
TCN (windows)	90,241	15.91 ± 1.01	11.97	11.54
XGBoost (engineered)	–	13.99 ± 0.46	9.81	9.88
Random Forest (engineered)	–	14.42 ± 0.48	9.87	10.98
XGBoost (raw)	–	16.42 ± 0.87	11.38	15.23

The networks never see the cycle number or an engineered feature, so the closest Phase 1 comparison is the raw feature set, where the best model reaches 16.42 from the current cycle's readings and the cycle number. All three networks do better, by 0.5 to 3.1 cycles, so the 30 cycle history carries information that a single cycle does not. For the Transformer that gain is larger than the 2.43 cycles the engineered features gave XGBoost. Close to failure, only the Transformer and the LSTM beat XGBoost on engineered features: on cycles with true RUL of at most 50 their RMSE is 7.98 and 8.84, against 9.88 for XGBoost and 11.54 for the TCN.

The search also shows three patterns, each from a few sampled configurations. Every one layer LSTM beat both sampled two layer LSTMs, but narrowly: 14.71 for the worst one layer configuration against 14.77 for the better two layer one. Both sampled four block TCNs beat all six three block TCNs by at least 1.4 cycles, but the fourth block adds depth and parameters as well as the last cycle of the window, so the search cannot say which of the two mattered. All three Transformers with dropout 0.3 were worse than all five with dropout 0.1.

The Transformer's lead over XGBoost is 0.71 cycles, and the two models' fold standard deviations are 0.34 and 0.46. Both models are the best of eight configurations on these folds, so both scores are slightly optimistic, and the test set gives the final ranking.

E. Learning curves

Fig. 13 shows the training and early stopping losses of each selected network on the first fold. For the LSTM and the Transformer the early stopping loss stops improving at epochs 11 and 10 while the training loss keeps falling slowly, but neither shows a sustained rise in early stopping loss. The TCN does overfit: after epoch 7 its training loss falls from 154 to 68 while its early stopping loss rises from 193 to between 210 and 355. Dropout of 0.1 does not prevent this. Early stopping limits the damage by restoring the epoch 7 weights. Over all 24 configurations no fold's best epoch is later than 35, so with a patience of 10 every run stops by epoch 45 of the allowed 100.

X. Discussion and Limitations

A. Features and learners

Both the feature set and the model family matter. Against the true test labels, engineered features lower the RMSE by 2.22 cycles for XGBoost, 2.54 for Random Forest and 4.30 for Elastic Net, while replacing Elastic Net with XGBoost lowers it by 4.99 on raw features and 2.91 on engineered features; cross validation shows the same pattern. The paired bootstrap interval for XGBoost on engineered minus raw features (-4.4 to -0.2 cycles) excludes zero, while the interval for Random Forest minus XGBoost (-1.3 to $+1.1$) does not, so once the features are fixed the choice between the two tree ensembles makes no measurable difference on 100 test engines. Our margin over the published Random Forest [6] comes mainly from the features: against labels clipped at 125 our Random Forest scores 16.58 on raw sensors and 14.02 on engineered features, against 17.91 published. The ablation of Section V places most of the feature gain on the rolling slopes, which give the rate of degradation rather than its level.

B. Accurate near failure, coarse far from it

Both ensembles are accurate where maintenance is decided. On the 33 test engines with true RUL of at most 50 cycles, the late-stage RMSE is 7.34 for Random Forest and 7.03 for XGBoost, against 17.88 and 18.11 on the 67 early-stage engines. Part of the early-stage error is built into the target: eleven test engines have a true RUL above the training cap, and no model on engineered features predicts above 130, so these engines are underestimated by construction. This error only brings maintenance forward. The costlier error appears between 51 and 75 cycles, where all three models predict too late (median error $+9$ to $+14$ cycles), around the point at which degradation first becomes visible in the sensors. Late predictions are the error that the score S penalises most.

C. Model selection and the test set

Cross validation selected XGBoost (RMSE 13.99 against 14.42 for Random Forest), while Random Forest is marginally better on the test set (RMSE 15.23 against 15.36, score 344 against 389). We kept the selection made before the test set was opened. The bootstrap interval above shows that the test ranking of the two ensembles is within sampling noise, and most of the score gap comes from a single engine that XGBoost predicts about 9 cycles earlier than Random Forest (Section VII). Cross validation and test numbers are not directly comparable: cross validation scores every cycle against the capped target, whereas the test set scores one cycle per engine against the true RUL. The cross validation scores of the selected configurations are also optimistic, because the same folds chose and scored them.

D. What the models rely on

Held-out permutation importance shows that both ensembles depend mainly on two inputs: the health index, whose shuffling raises RMSE by about 11 cycles for XGBoost and 7.4 for Random Forest, and the cycle number (4.84 and 5.22). No individual sensor group adds more than 1.62. The sensor information therefore reaches the models largely through the health index, a single learned combination of all 14 sensors. Reliance on the cycle number means the models partly predict from age. This is useful on FD001, where every engine starts in normal condition and training lifetimes span 128 to 362 cycles, but it ties the models to the lifetime distribution of this fleet. Impurity and permutation importance also disagree for correlated inputs: `s_4` receives 26% of Random Forest's impurity reduction but costs only 1.62 cycles when shuffled, so impurity rankings alone would overstate its role.

E. Gap to deep sequence models

Four published deep models are more accurate than our Random Forest and XGBoost by 0.8 to 1.5 cycles of RMSE against clipped labels, and on the score S six published models do better than our Random Forest and eight better than the selected XGBoost. These models learn features from windows of the raw sensor sequence. Our features look back at most 20 cycles and use window lengths chosen by hand, so they cannot represent how long a sensor has been drifting or when its drift began to accelerate. The remaining gap is consistent with this limit, although our experiments do not isolate it.

F. Limitations

All results are on FD001, which has a single operating condition and a single fault mode. The regime normalisation of Section V reduces to a z-score here and has not been tested on the multi-condition subsets. The test set has 100 engines with one prediction each, so the test metrics carry wide sampling intervals, as the bootstrap shows; the late stage contains only 33 engines. The cap of 125 cycles bounds what the models can predict, so engines with more remaining life are always underestimated. Published FD001 results differ in protocol details that are not always reported, such as whether test labels are clipped and the window length used, so the comparison with them is approximate. Finally, C-MAPSS is simulated, and conclusions about which sensors matter apply to the simulation rather than to physical engines.

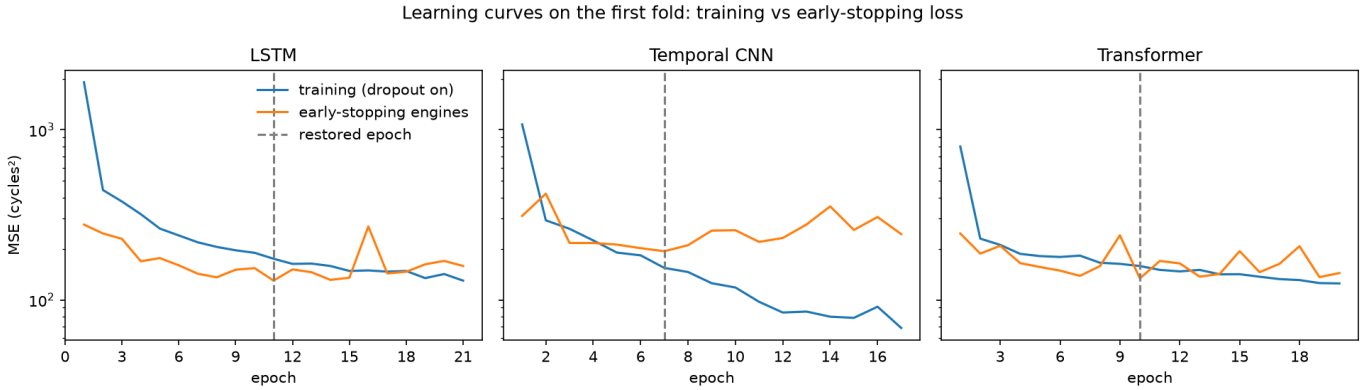


Fig. 13. Training loss (dropout active) and early stopping loss of each selected network on the first fold. The dashed line marks the epoch whose weights are restored.

XI. Conclusion and Phase 2 Plan

We predicted the remaining useful life of FD001 engines with engineered degradation features and three classical regressors, under a protocol that splits by engine, refits feature statistics inside every fold and scores the test set once. The selected XGBoost model reaches a test RMSE of 15.36 cycles overall and 7.03 cycles on engines within 50 cycles of failure. Against labels clipped at 125, Random Forest and XGBoost reach 14.02 and 14.08, ahead of published Random Forest and gradient boosting results and 0.8 to 1.5 cycles behind the best published deep models. The engineered features and the tree ensembles both improve on the linear models with raw sensors, the two tree ensembles cannot be separated on the test set, and both rely mainly on the health index and the cycle number.

Phase 2 replaces the fixed 5-, 10- and 20-cycle window summaries with models that read the sensor sequence directly: an LSTM, a temporal convolutional network or a Transformer encoder on fixed-length windows of the raw sensors. It adds an ablation study and a hybrid model that feeds the engineered features into the sequence model, which tests whether the hand-made features still help once the history is learned. The sequence models will be evaluated under the protocol of this report: FD001, five-fold cross validation grouped by engine, training RUL capped at 125, one prediction per test engine scored once, and the same RMSE, score S and early and late stage metrics, so that their results are directly comparable with Table V.

XII. Contributions

- **Sarvesh Srinath**: repository setup, working conventions, pinned dependencies, continuous integration and the pipeline that reruns every notebook; the engine-level split and evaluation metrics; the final test-set evaluation, error analysis and comparison with published results; reproducibility checks; review and merging of pull requests. Wrote Sections IV and VII.
- **Trishit Swarnakar**: loading of FD001 and construction of the RUL target; exploratory data analysis, sensor selection and the choice of target cap. Wrote Section III.
- **Aryan Kumar**: degradation feature engineering (regime normalisation, rolling statistics, rolling slopes and health index) and the feature ablation. Wrote Section V.
- **Husain Khorakiwala**: model implementation, baselines and hyperparameter tuning with engine-grouped cross validation; the RUL cap experiment; feature importance and SHAP analysis; review of the test-set evaluation and reproducibility work. Wrote Section VI.
- **Navodit Sharma**: literature review; assembly of the report; presentation slides. Wrote the abstract and Sections I, II, X and XI.

References

[1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *2008 International Conference on Prognostics and Health Management*, 2008, pp. 1–9.

[2] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in *2008 International Conference on Prognostics and Health Management*, 2008, pp. 1–6.

[3] G. S. Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in *International Conference on Database Systems for Advanced Applications*, 2016, pp. 214–228.

[4] M. Ragab, Z. Chen, M. Wu, C.-K. Kwoh, R. Yan, and X. Li, "Attention sequence to sequence model for machine remaining useful life prediction," arXiv:2007.09868, 2020.

[5] T. Wang, J. Yu, D. Siegel, and J. Lee, "A similarity-based prognostics approach for remaining useful life estimation of engineered systems," in *2008 International Conference on Prognostics and Health Management*, 2008, pp. 1–6.

[6] C. Zhang, P. Lim, A. K. Qin, and K. C. Tan, "Multiobjective deep belief networks ensemble for remaining useful life estimation in prognostics," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 28, no. 10, pp. 2306–2318, 2017.

[7] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)*, 2017, pp. 88–95.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[9] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," *Reliability Engineering & System Safety*, vol. 172, pp. 1–11, 2018.

[10] C. Zhang, "Computational intelligence in diagnostic and prognostic applications," Ph.D. dissertation, National University of Singapore, 2017. [Online]. Available: https://scholarbank.nus.edu.sg/handle/10635/142804

[11] Q. Wang, S. Zheng, A. Farahat, S. Serita, and C. Gupta, "Remaining useful life estimation using functional data analysis," arXiv:1904.06442, 2019.

[12] L. Liu, L. Wang, and Z. Yu, "Remaining useful life estimation of aircraft engines based on deep convolution neural network and LightGBM combination model," *International Journal of Computational Intelligence Systems*, vol. 14, no. 1, p. 165, 2021.

[13] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 67, no. 2, pp. 301–320, 2005.

[14] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[15] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[16] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[17] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[18] S. M. Lundberg, G. Erion, H. Chen, A. DeGrave, J. M. Prutkin, B. Nair, R. Katz, J. Himmelfarb, N. Bansal, and S.-I. Lee, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.

[19] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.

[20] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[22] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.